



I118 - Taller de Proyecto II

Informe de Avance - Noviembre

Proyecto A5 - Mano simulada DMP

Grupo de desarrollo

Acuña, Lucia - 03213/1

Avila Montoya, Eygleen Fernanda - 02931/2

Bejarano, Abril - 03339/5

1. Descripción general del proyecto

El proyecto tiene como objetivo comparar dos métodos de estimación de orientación tridimensional. Para esto, se está desarrollando un sistema de adquisición y visualización de orientación a partir de los datos obtenidos de una unidad de medición inercial (IMU). El primer método utiliza la orientación calculada por el procesador digital de movimiento (DMP) del sensor y el segundo emplea filtros implementados por software sobre las mediciones del acelerómetro, el giroscopio y el magnetómetro.

Inicialmente se planteó utilizar un sensor MPU9250 en lugar de un MPU6050 para contar con los datos del magnetómetro, potencialmente útiles como referencia para el eje de yaw. Sin embargo, debido a la dificultad para conseguir este sensor y a que las unidades disponibles no funcionaron correctamente, se decidió volver a la propuesta original y utilizar un MPU6050, complementando sus mediciones con un magnetómetro QMC5883L.

El dispositivo final estará compuesto por ambos sensores montados sobre un guante y conectados por I²C a un ESP32. Los resultados se transmitirán a una interfaz web que mostrará dos modelos tridimensionales y una gráfica de diferencias entre estimaciones.

2. Materiales y presupuesto

En la Tabla 1 se presenta la lista de materiales y presupuesto del proyecto.

Tabla 1: Materiales necesarios para el desarrollo del prototipo y su disponibilidad.

Cantidad	Descripción	Disponible	Precio unitario
1	ESP32 NodeMCU WROOM-32, 38 pines, USB-C, Wi-Fi y Bluetooth	Sí	$20.700
1	MPU6050	Sí	$6.000
1	Magnetómetro QMC5883L	Sí	$7.266
1	Guante para montaje del sistema	Pendiente	A definir
1	Juego de cables Dupont x10	Sí	—
1	Capacitor de desacople de 0,1 µF	Sí	—
1	Cable MicroUSB para alimentación y programación	Sí	—
1	Batería para alimentación portátil	Sí	—
1	Placa perforada para el montaje de componentes	Sí	$4000
2	Tiras de pines hembra	Sí	$2000
1	Tira de pines macho	Sí	$1000
		Total	**$40.966**

La cátedra proporcionó el ESP32 y las dos unidades del módulo MPU9250 que se habían probado anteriormente. El MPU6050 y el magnetómetro QMC5883L fueron adquiridos por el

equipo de desarrollo para realizar los ensayos y, tras el cambio de propuesta, pasan a ser los sensores definitivos.

Los demás materiales ya estaban disponibles, por lo que solo resta adquirir el guante sobre el cual se montará el sistema.

En la propuesta inicial se contemplaba el uso de tres capacitores de desacople. Sin embargo, durante el montaje se comprobó que dos de ellos correspondían a pines que no se utilizan en este proyecto, por lo que la cantidad se redujo a uno. Esta decisión deberá verificarse al armar el prototipo final, ya que es posible que no se requiera ninguno.

El resto de los materiales se mantiene según lo establecido en el plan de proyecto. Con el cambio de sensores, el costo total se redujo de $48.660 a $40.966, incluso después de la adquisición de componentes para el montaje del sistema.

3. Esquema gráfico del proyecto

La arquitectura del sistema separa las etapas de adquisición, procesamiento y visualización, tal como se presentó en el Plan de Proyecto (Figura 1). El ESP32 recibe las mediciones por I²C, estima la orientación mediante los dos métodos previstos y transmite los resultados a la interfaz de visualización.

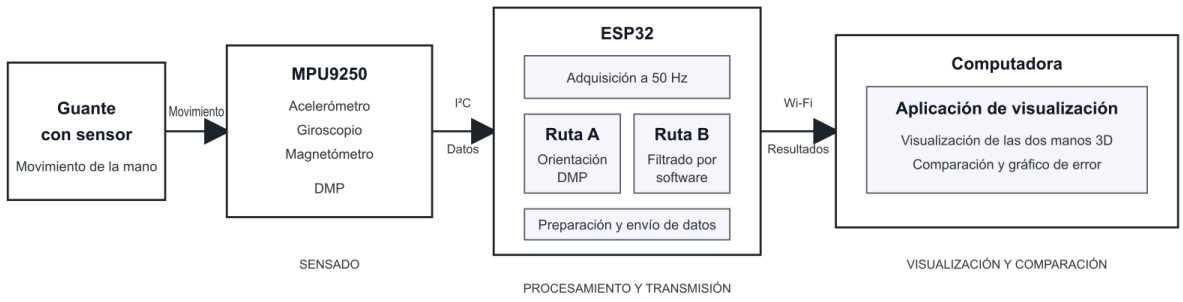


Figura 1: Diagrama de bloques del sistema presentado en el Plan de Proyecto.

El prototipo final mantiene esta estructura, a excepción de los sensores. Sobre el ESP32 se ejecutan en paralelo las dos rutas de estimación, con una frecuencia de adquisición de 50 Hz. Los resultados se envían a la computadora por Wi-Fi, reemplazando la conexión USB serie implementada durante las pruebas de octubre. La interfaz web muestra los dos modelos tridimensionales de la mano y un gráfico con la diferencia entre ambas estimaciones. El esquema resultante se presenta en la Figura 2.

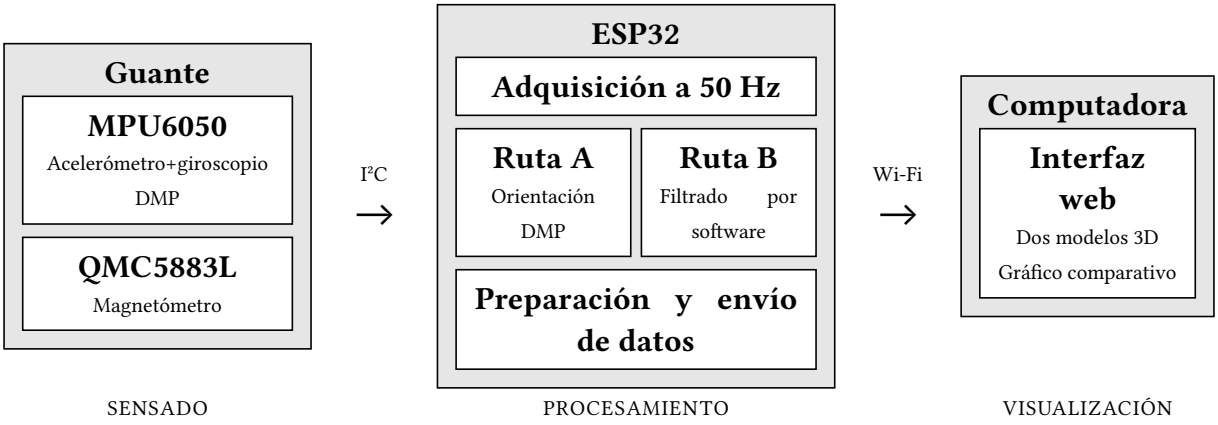


Figura 2: Diagrama de bloques actual.

4. Grado de avance, problemas y soluciones

En la primera etapa del proyecto se completaron 5 de las 6 tareas planificadas: la investigación teórica y de librerías, la adquisición y conexión del hardware, la comunicación I²C y lectura de datos (tanto crudos como los del DMP), la configuración inicial y la implementación del filtro complementario. Quedaron pendientes para esta etapa el montaje del hardware y la transmisión Wi-Fi del ESP32 a la PC.

Para la segunda etapa, el plan de proyecto preveía completar la calibración del DMP, implementar el filtro de Kalman, refinar el filtro complementario, desarrollar la interfaz con visualización 3D, el selector de filtros y el gráfico de error, e integrar el prototipo final (Tabla 2).

Tabla 2: Diagrama de Gantt de la Etapa 2.

Fase / Tarea planificada	Octubre			Noviembre
	12	19	26	2
Etapa 2: Integración, Filtro de Kalman y visualización 3D				
Calibración fina de offsets del DMP	■			
Implementación y ajuste del Filtro de Kalman	■	■		
Desarrollo visual (manos 3D)	■	■	■	
Optimización del Filtro Complementario		■	■	
Panel selector y graficador de error		■	■	
Integración de prototipo funcional (50 Hz)				■
Redacción del Informe de Avance E3				■
Entrega formal (hito)				
Informe de Avance 2 (E3) – 08/11/2026				◆

En las siguientes secciones se evalúa el grado de avance de cada uno de estos items.

4.1. Montaje de hardware

Se mantuvo la conexión utilizada en el informe de avance de octubre, la cual se muestra en la Figura 3, agregando la conexión a los pines VIN y GND del otro lado del ESP32 para conectar la batería.

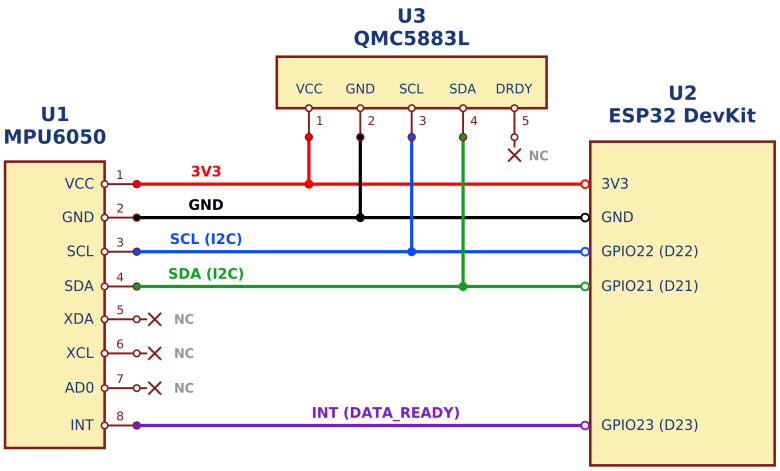


Figura 3: Esquema de arquitectura general del sistema.

El montaje sobre el guante no se realizó y queda pendiente para la próxima etapa. Sí se comenzó el armado de los componentes sobre una placa perforada, utilizando tiras de pines hembra para poder retirar los sensores y el ESP32 con facilidad. Todavía no se realizaron las soldaduras de las conexiones.

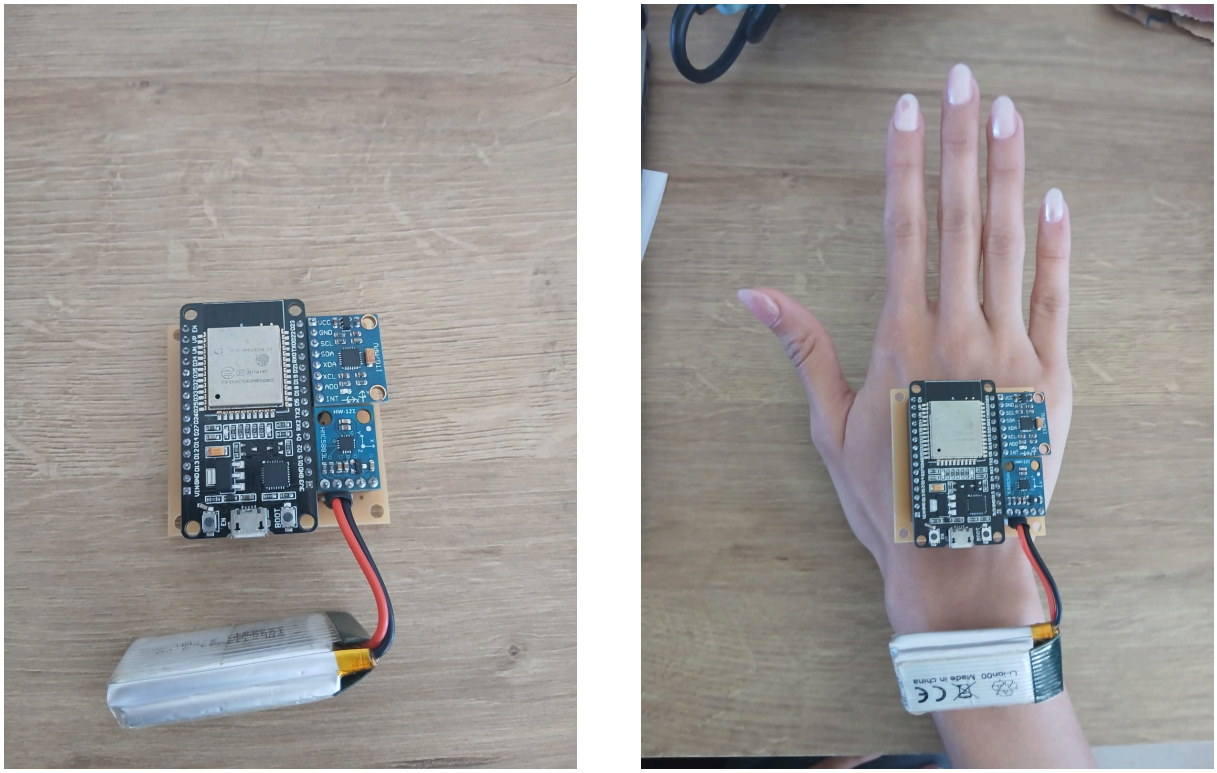


Figura 4: Avance del montaje de hardware.

4.2. Transmisión Wi-Fi del ESP32 a la PC

Durante esta etapa se completó la transmisión inalámbrica de las dos orientaciones, que en el informe anterior había quedado pendiente y se reemplazaba por una conexión USB serie. El

ESP32 envía los resultados a la interfaz web por Wi-Fi, y la página puede además controlar el firmware mediante comandos.

4.2.1. Conexión

Para la transmisión se utilizó WebSocket, que mantiene una única conexión abierta por la que el ESP32 envía mensajes de texto a medida que están disponibles. Frente a un esquema de pedidos HTTP, esto evita abrir una conexión por cada dato y reduce la latencia. Se empleó la librería `WebSocketsServer`, con el ESP32 como servidor en el puerto 81.

El ESP32 trabaja en modo Access Point más estación (`WIFI_AP_STA`). Por un lado, crea su propia red (`ManoDMP`), que siempre queda activa y permite usar el sistema sin depender de un router. Por otro, si existe un archivo `secretos.h` con las credenciales de una red local, se conecta a ella con una dirección IP fija y se anuncia por mDNS como `manodmp.local`. Además, se desactivó el ahorro de energía del Wi-Fi para reducir la latencia (Código 1).

```
1  static void wifiIniciar() {
2    bool usarRed = strlen(RED_SSID) > 0;
3    WiFi.mode(usarRed ? WIFI_AP_STA : WIFI_AP);
4    WiFi.softAP(AP_SSID, AP_PASS);            // red propia, siempre activa
5
6    if (usarRed) {
7      IPAddress local_IP(192, 168, 0, 11);    // IP fija en la red local
8      IPAddress gateway(192, 168, 0, 1);
9      IPAddress subnet(255, 255, 255, 0);
10     WiFi.config(local_IP, gateway, subnet);
11     WiFi.begin(RED_SSID, RED_CLAVE);
12
13     uint32_t t0 = millis();
14     while (WiFi.status() != WL_CONNECTED && millis() - t0 < TIMEOUT_RED_MS) {
15       delay(250);
16     }
17   }
18   WiFi.setSleep(false); // reduce la latencia
19
20   if (usarRed && WiFi.status() == WL_CONNECTED && MDNS.begin(NOMBRE_MDNS)) {
21     MDNS.addService("ws", "tcp", PUERTO_WS); // ws://manodmp.local:81
22   }
23   ws.begin();
24   ws.onEvent(eventoWs);
25 }
```

Código 1: Inicio del Wi-Fi, Access Point, conexión a la red local y servidor WebSocket.

Si la conexión a la red local falla dentro del tiempo de espera de 10 segundos, el equipo continúa funcionando solo con su red propia.

4.2.2. Formato y frecuencia de los mensajes

Cada mensaje es un objeto JSON con un contador de muestras (`n`), el timestamp del ESP32 en milisegundos (`t`), el filtro activo (`f`) y las dos orientaciones como cuaterniones `[w, x, y, z]`, la del DMP (`dmp`) y la estimada por software (`sw`). Se eligieron cuaterniones porque el DMP los entrega de forma nativa y porque simplifican la comparación entre ambas estimaciones en la interfaz. Como la estimación por software se calcula en ángulos de Euler, se convierte a cuaternión antes de enviarla. El contador permite que la página detecte mensajes perdidos (Código 2).

```
static void enviarMuestra(const Quaternion& qDmp) {
  if (!transmitiendo || ws.connectedClients() == 0) return;

  float r, p, y; const char* nombre;
  elegirEstimacion(r, p, y, nombre);        // según el filtro elegido
  float s[4];
  cuatDesdeEuler(r, p, y, s);               // estimación por software

  char buf[200];
  int n = snprintf(buf, sizeof(buf),
    "{\"n\":%lu,\"t\":%lu,\"f\":\"%s\","
    "\"dmp\":[%.5f,%.5f,%.5f,%.5f],\"sw\":[%.5f,%.5f,%.5f,%.5f]}",
    (unsigned long) contadorEnvio++, (unsigned long) millis(), nombre,
    qDmp.w, qDmp.x, qDmp.y, qDmp.z, s[0], s[1], s[2], s[3]);
  ws.broadcastTXT(buf, n);
}
```

Código 2: Armado y envío de cada mensaje por WebSocket.

El DMP entrega paquetes a aproximadamente 100 Hz. Para alcanzar la frecuencia objetivo de 50 Hz, solo se transmite uno de cada dos paquetes (Código 3), aunque los filtros procesan todos para integrar con mayor resolución . Además, si no hay ningún cliente conectado o la transmisión está pausada, no se arma el mensaje para no gastar tiempo de procesamiento.

```
static const uint8_t DIVISOR_ENVIO = 2;

// En procesarPaquete(), luego de actualizar los filtros:
if (++contadorPaquetes % DIVISOR_ENVIO == 0) enviarMuestra(q);
```

Código 3: Submuestreo del envío a 50 Hz.

4.2.3. Comandos desde la página

La conexión es bidireccional: la interfaz envía comandos JSON cortos para iniciar o detener la transmisión, elegir el filtro (`comp`, `mag` o `kalman`) y pedir la calibración del sensor o del magnetómetro. Para no depender de otra librería, el firmware busca las palabras clave en el texto recibido (Código 4).

Algunos de estos comandos implican tareas lentas. La calibración del MPU6050, por ejemplo, ejecuta `CalibrateAccel` y `CalibrateGyro`, que tardan varios segundos. Si se realizara dentro del evento de red, el ESP32 dejaría de atender el WebSocket durante ese tiempo, se detendría el envío de muestras y la conexión podría cerrarse por inactividad. Para evitarlo, el manejador de comandos solo activa un flag y retorna de inmediato (Código 4).

```
bool pedirCalibracion = false;   // flag de calibración

static void manejarComando(const char* txt) {
  if (strstr(txt, "\"iniciar\""))          transmitiendo = true;
  else if (strstr(txt, "\"detener\""))     transmitiendo = false;
  else if (strstr(txt, "\"calibrarMag\"")) magComenzarCalibracion();
  else if (strstr(txt, "\"calibrar\""))    pedirCalibracion = true; // no se calibra acá
  else if (strstr(txt, "\"filtro\"")) {
    if (strstr(txt, "\"comp\""))           modo = MODO_COMP;
    else if (strstr(txt, "\"mag\""))       modo = MODO_MAG;
    else if (strstr(txt, "\"kalman\""))    modo = MODO_KALMAN;
  }
}
```

Código 4: El manejador de comandos solo marca la tarea pendiente.

La tarea se atiende después, desde el bucle principal. En cada iteración de `loop()` se llama primero a `ws.loop()`, que procesa los eventos de red y dispara `manejarComando()`. A continuación se revisa la bandera y, si está activa, se la baja y se ejecuta la recalibración. De esta forma, la parte lenta no se realiza nunca dentro del callback de la comunicación (Código 5).

```
void loop() {
  ws.loop();                      // atiende la red (puede activar flags)

  if (pedirCalibracion) {         // la tarea se ejecuta fuera del evento de red
    pedirCalibracion = false;
    recalibrar();
  }

  if (!paqueteListo) return;
  paqueteListo = false;
  if (mpu.dmpGetCurrentFIFOPacket(fifoBuffer)) procesarPaquete();
}
```

Código 5: Atención de la bandera en el bucle principal.

El mismo criterio se aplica a la interrupción del DMP. La rutina `isrDmp()` solo pone `paqueteListo = true` y el procesamiento real se hace en `loop()`.

Mientras dura la recalibración, el equipo deja de enviar muestras, ya que el DMP se desactiva y se reinicia su FIFO. Al terminar, el filtro vuelve a arrancar desde la posición real del sensor (Código 6).

```
static void recalibrar() {
  mpu.setDMPEnabled(false);
  calibrarSensor();              // CalibrateAccel(6) y CalibrateGyro(6)
  mpu.resetFIFO();
  mpu.setDMPEnabled(true);
  filtroIniciado = false;        // el filtro arranca de nuevo
}
```

Código 6: Recalibración: se reinicia el DMP y el filtro.

La transmisión quedó integrada al firmware y la interfaz recibe ambas orientaciones por Wi-Fi.

4.3. Calibración de offsets del DMP

Los sensores inerciales presentan un error de offset. Con el dispositivo en reposo, el giroscopio no mide exactamente cero y el acelerómetro no mide exactamente (0, 0, 1 g). Al integrar la velocidad angular, esta diferencia se acumula como una deriva creciente en el ángulo.

Hasta el informe anterior los offsets del MPU6050 se fijaban a mano con valores aproximados. En esta etapa se incorporó una calibración automática de los offsets del acelerómetro y del giroscopio, que se ejecuta al iniciar el equipo.

El MPU6050 permite cargar un offset por eje y por sensor en registros internos, que se restan de la medición antes de que esta llegue al DMP y a la FIFO. Se aprovechó esta característica y la calibración se realiza sobre esos registros, en lugar de restar un sesgo por software. De este modo la corrección alcanza a las dos rutas de estimación (Código 7).

```
static void calibrarSensor() {
  mpu.CalibrateAccel(6);
  mpu.CalibrateGyro(6);
}
```

Código 7: Calibración de los offsets del acelerómetro y del giroscopio.

Las funciones `CalibrateAccel` y `CalibrateGyro` de la librería promedian lecturas del sensor, ajustan el offset del registro correspondiente y repiten el proceso hasta que la medición converge al valor esperado en reposo. Mientras se está calibrando el guante debe permanecer quieto y apoyado de modo que el eje Z quede alineado con la gravedad. El parámetro (6) indica la cantidad de ciclos de refinamiento.

La calibración de offsets quedó integrada al arranque y queda para la siguiente etapa incorporar un botón en la interfaz web

4.4. Implementación y ajuste del filtro de Kalman

El filtro complementario combina el giroscopio y la referencia (acelerómetro o magnetómetro) con un peso fijo, `ALPHA`, que no cambia con las condiciones de medición. El filtro de Kalman, en cambio, calcula en cada paso cuánto confiar en cada fuente a partir de un modelo del error, y además estima el sesgo del giroscopio en línea. Esto lo hace menos dependiente de una buena calibración inicial y, en principio, más robusto ante la deriva.

Hasta el informe anterior la estimación por software se limitaba al filtro complementario. En esta etapa se implementó un filtro de Kalman que corre en el ESP32.

Para cada eje se implementó un filtro de Kalman de una dimensión, con dos variables de estado: el ángulo θ y el sesgo del giroscopio b. El giroscopio se usa como entrada del modelo y la referencia absoluta (acelerómetro para roll y pitch, magnetómetro para yaw) como medición.

En la predicción, el ángulo se integra con la velocidad angular medida, corregida por el sesgo estimado, y el sesgo se supone constante:

$$\theta_k = \theta_{k-1} + (\omega_k - b_{k-1}) \cdot \Delta t, \quad b_k = b_{k-1} \tag{1}$$

En la corrección, la medición z es el ángulo de referencia. Como esta solo observa el ángulo y no el sesgo, la ganancia de Kalman K tiene dos componentes: una corrige el ángulo y la otra el sesgo.

$$K = P\frac{H^T}{HPH^T + R}, \quad H = (1 \;\; 0) \tag{2}$$

$$\theta \leftarrow \theta + K_0(z - \theta), \quad b \leftarrow b + K_1(z - \theta) \tag{3}$$

Aquí P es la matriz de covarianza del error de estimación y R el ruido de la medición. Cada eje se implementó como una estructura independiente con su propio estado y su propia covarianza (Código 8).

```
struct Kalman1D {
  float angulo, bias, rate;
  float P[2][2];                // covarianza del error de estimación
  float Q_angle, Q_gyro;        // ruido de proceso
  float R_measure;              // ruido de medición

  float getAngle(float newAngle, float newRate, float dt) {
    // Predicción: integrar el giroscopio sin el sesgo estimado
    rate = newRate - bias;
    angulo += dt * rate;
    P[0][0] += dt * (dt * P[1][1] - P[0][1] - P[1][0] + Q_angle);
    P[0][1] -= dt * P[1][1];
    P[1][0] -= dt * P[1][1];
    P[1][1] += Q_gyro * dt;

    // Corrección: ganancia de Kalman e innovación
    float S = P[0][0] + R_measure;
    float K0 = P[0][0] / S;
    float K1 = P[1][0] / S;
    float y = newAngle - angulo;
    angulo += K0 * y;
    bias   += K1 * y;

```

```
    // Actualizar la covarianza
    float P00 = P[0][0], P01 = P[0][1];
    P[0][0] -= K0 * P00;  P[0][1] -= K0 * P01;
    P[1][0] -= K1 * P00;  P[1][1] -= K1 * P01;
    return angulo;
  }
};
```

Código 8: Filtro de Kalman de una dimensión (ángulo y sesgo del giroscopio).

Se creó una instancia por eje. Para roll y pitch la medición es el ángulo de inclinación calculado con el acelerómetro, igual que en el filtro complementario. Para el yaw la medición es el rumbo del magnetómetro, compensado por inclinación con el roll y el pitch ya estimados (Código 9).

El yaw requiere un tratamiento especial por ser un ángulo circular: la diferencia entre la medición y la estimación podría salir como cercana a 2π al cruzar $\pm 180°$, lo que produciría un salto brusco. Para evitarlo, la medición se expresa como la estimación actual más la diferencia normalizada al rango $[-\pi, \pi]$, y el resultado se vuelve a normalizar.

```
static void kalmanActualizar(float ax, float ay, float az,
                             float gx, float gy, float gz, float dt) {
  // Referencias absolutas
  float rollAcc   = atan2f(ay, az);
  float pitchAcc  = atan2f(-ax, sqrtf(ay * ay + az * az));
  float yawMagRef = yawDesdeMag(kRoll, kPitch);   // con compensación de inclinación

  kRoll  = kalmanRoll.getAngle(rollAcc, gx, dt);
  kPitch = kalmanPitch.getAngle(pitchAcc, gy, dt);

  // Yaw: la innovación se normaliza para evitar saltos en ±PI
  float diffYaw = normalizarAngulo(yawMagRef - kYaw);
  kYaw = normalizarAngulo(kalmanYaw.getAngle(kYaw + diffYaw, gz, dt));
}
```

Código 9: Actualización de los tres filtros de Kalman en cada paquete.

4.4.1. Ajuste de parámetros

El comportamiento del filtro queda determinado por tres parámetros (Código 10):

- `Q_angle`: ruido de proceso del ángulo. Un valor mayor indica menor confianza en el modelo y hace que el filtro siga más rápido a la referencia.
- `Q_gyro`: ruido de proceso del sesgo. Un valor mayor permite que el sesgo estimado cambie más rápido.
- `R_measure`: ruido de medición. Un valor mayor indica menor confianza en el acelerómetro o el magnetómetro, por lo que el filtro se apoya más en el giroscopio.

```
void init() {
  angulo = 0.0f;  bias = 0.0f;
```

```
3    P[0][0] = P[0][1] = P[1][0] = P[1][1] = 0.0f;
4    Q_angle   = 0.001f;
5    Q_gyro    = 0.003f;
6    R_measure = 0.03f;
7  }
```

Código 10: Parámetros de ruido del filtro de Kalman.

El filtro de Kalman quedó integrado al firmware, y queda para la siguiente etapa ajustarlo para mejorar la aproximación. Actualmente, los filtros arrancan con el ángulo en cero en lugar de la posición real del sensor, y no se reinician al recalibrar, por lo que tardan un tiempo en converger.

4.5. Optimización del filtro complementario

El filtro complementario de la etapa anterior corregía solo roll y pitch con el acelerómetro, por lo que el yaw derivaba sin límite. En esta etapa se le agregó una corrección del yaw con el magnetómetro en otra variante seleccionable desde la interfaz para poder compararlas.

El rumbo magnético se calcula compensando la inclinación con el roll y el pitch ya estimados, de modo que una inclinación del guante no se confunda con un giro. Luego se fusiona con el giroscopio igual que en roll y pitch, pero corrigiendo la diferencia normalizada al rango $[-\pi, \pi]$, para evitar saltos al cruzar $\pm 180°$ (Código 11).

```
1  // Variante comp: solo giroscopio (deriva, es lo esperado)
2  fYawComp = normalizarAngulo(fYawComp + gz * dt);
3
4  // Variante mag: giroscopio corregido con el rumbo magnético
5  float yawGiro = fYawMag + gz * dt;
6  float dif = normalizarAngulo(yawDesdeMag(fRoll, fPitch) - yawGiro);
7  fYawMag = normalizarAngulo(yawGiro + (1.0f - ALPHA) * dif);
```

Código 11: Fusión del yaw con el rumbo del magnetómetro.

Para evitar errores en el arranque y en la integración se incorporaron las siguientes mejoras:

- El filtro no parte de cero: en la primera muestra se inicializa con la inclinación del acelerómetro y el rumbo del magnetómetro, y se reinicia al recalibrar.
- Se descartan los intervalos `dt` no válidos (nulos o mayores a 0,1 s), que aparecen tras una pausa y distorsionarían la integración.
- El filtro procesa todos los paquetes del DMP (100 Hz), aunque se transmita solo uno de cada dos, lo que mejora la integración del giroscopio (Código 12).

```
1  if (!filtroIniciado) { filtroIniciar(ax, ay, az); filtroIniciado = true; return; }
2  if (dt <= 0.0f || dt > 0.1f) return;   // pausa larga: descartar
3  filtroComplementario(ax, ay, az, gx, gy, gz, dt);
```

Código 12: Inicialización y descarte de intervalos inválidos.

El peso del giroscopio se fijó en `ALPHA = 0.98`. Un valor mayor reduce el ruido del acelerómetro pero corrige más lento la deriva, y uno menor hace lo contrario. [

4.6. Interfaz web

La interfaz (Figura 5) es una página estática (HTML, CSS y JavaScript) que se abre en el navegador de la PC y se conecta por WebSocket al ESP32. Para el modelo 3D se usó Three.js y para los gráficos Chart.js. Incluye además un modo demo que simula el flujo de datos a 50 Hz, lo que permitió desarrollar y probar la interfaz sin el hardware. En el panel de enlace de datos se muestra la frecuencia real de recepción y los paquetes perdidos, calculados con el contador `n` de cada mensaje.

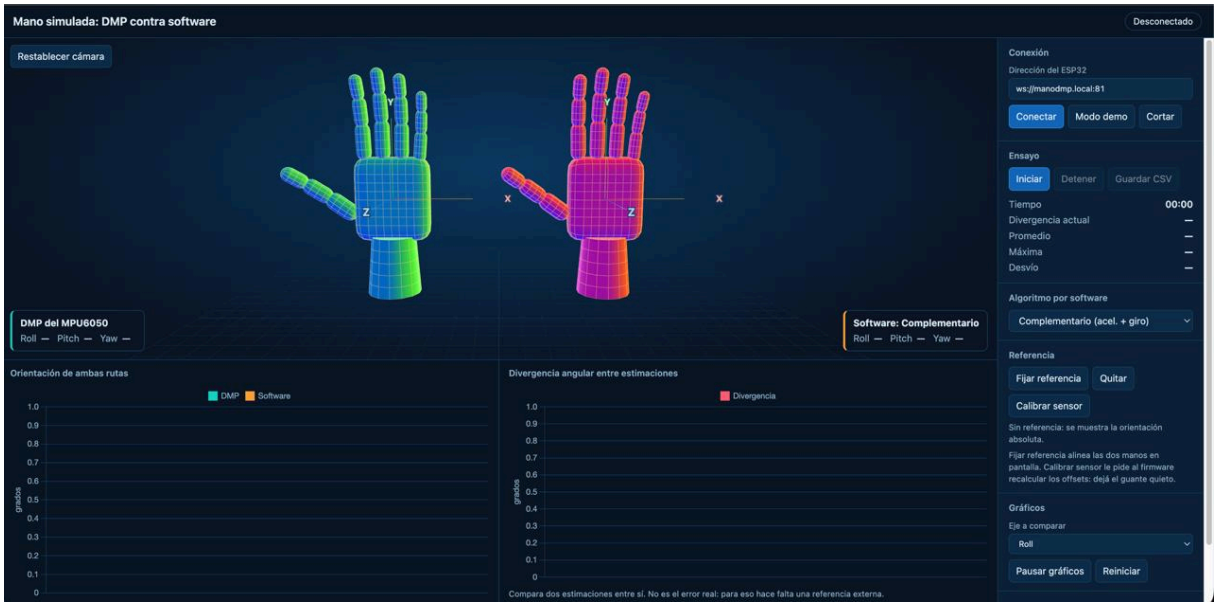


Figura 5: Avances en la interfaz web.

4.6.1. Visualización 3D

Se muestran dos manos lado a lado, la de la izquierda sigue la orientación del DMP y la de la derecha la estimación por software. Los modelos se construyen por código con primitivas (caja redondeada, cilindro y cápsulas) y un material con degradé y grilla, con una paleta distinta para cada uno. Los dedos tienen una postura fija, ya que solo se representa la orientación global. Cada mano lleva dibujados los ejes del sensor (X hacia el meñique, Y hacia los dedos y Z saliendo del dorso), de modo que se vea hacia dónde apunta cada eje.

Como ambas estimaciones llegan como cuaterniones, se aplican directamente al modelo, sin pasar por ángulos de Euler. Los ejes del modelo coinciden con los del sensor, por lo que no se cambian ejes ni signos. Además, en cada cuadro la orientación mostrada se acerca a la recibida con una interpolación esférica (`slerp`), lo que suaviza el movimiento a costa de un retardo visual pequeño. Los gráficos usan los datos sin suavizar (Código 13).

```
1 const sensorAThree = q => new THREE.Quaternion(q[1], q[2], q[3], q[0]);
2 objetivo.dmp.copy(sensorAThree(q.dmp));
3 objetivo.sw.copy(sensorAThree(q.sw));
4 manos[k].quaternion.slerp(objetivo[k], 0.4);
```

Código 13: Aplicación del cuaternión recibido a cada modelo.

Como las dos estimaciones pueden arrancar con orientaciones absolutas distintas (el yaw del DMP y el del magnetómetro no tienen por qué coincidir), el botón *Fijar referencia* guarda la orientación actual de cada ruta y desde ese momento se muestra la rotación relativa a ella, de modo que las dos manos parten alineadas. La cámara es orbitable con el mouse y puede restablecerse con un botón.

4.6.2. Panel selector

El panel lateral muestra los controles para conectar al ESP32 (con la dirección `ws://manodmp.local:81` por defecto) o iniciar el modo demo, iniciar y detener el ensayo, fijar o quitar la referencia, pedir la calibración y elegir el eje que se grafica (roll, pitch o yaw). Un desplegable permite elegir el algoritmo por software entre `comp` (filtro complementario con acelerómetro y giroscopio), `mag` (filtro complementario con los 3 sensores) y `kalman` (filtro Kalman con los 3 sensores). Al cambiarlo, la página envía el comando al ESP32 y actualiza el rótulo de la mano de software, sin reiniciar el equipo (Código 14).

```
$('filtro').onchange = e => {
  con.filtro(e.target.value);                    // "comp", "mag" o "kalman"
  $('filtroNombre').textContent = e.target.selectedOptions[0].text;
};

// conexion.js
filtro(f) { filtro = f; api.enviar({ cmd: 'filtro', valor: f }); }
```

Código 14: Selección del filtro desde la interfaz.

4.6.3. Gráfico de error

Se muestran dos gráficos con una ventana de las últimas 400 muestras (unos 8 segundos a 50 Hz). El primero superpone la orientación de ambas rutas en el eje elegido, lo que permite ver, por ejemplo, la deriva del yaw. Como los ángulos de Euler saltan al cruzar $\pm 180°$, antes de graficar cada valor se desplaza un múltiplo de 360° para que quede lo más cerca posible del anterior, y así la curva no tenga saltos falsos.

El segundo gráfico muestra la diferencia angular entre las dos estimaciones, definida como el ángulo de la rotación que lleva una orientación a la otra. Para dos cuaterniones unitarios q_1 y q_2 es

$$\theta = 2 \arccos(|q_1 \cdot q_2|) \tag{4}$$

El valor absoluto evita que q y $-q$, que representan la misma rotación, den un valor falso, y como se calcula con los cuaterniones no tiene los problemas de los ángulos de Euler (Código 15).

```
// Ángulo de la rotación que lleva una orientación a la otra
export const angulo = (a, b) =>
  2 * Math.acos(Math.min(1, Math.abs(a[0]*b[0] + a[1]*b[1] + a[2]*b[2] + a[3]*b[3]))) *
  D;
```

```
4
5 // Evita saltos de 360° al graficar un ángulo en el eje elegido
6 const unir = (a, v) =>
7   a.length ? v + 360 * Math.round((a[a.length - 1] - v) / 360) : v;
```

Código 15: Divergencia angular y continuidad del ángulo graficado.

4.7. Otros avances

Aunque no estaba previsto en el Plan de Proyecto, fue necesario agregar una calibración antes de poder usar el magnetómetro. Al incorporar el QMC5883L se comprobó que sus lecturas crudas no servían como referencia de yaw ya que el campo magnético medido está desplazado y deformado por el hierro y la electrónica cercanos al sensor (efectos *hard-iron* y *soft-iron*).

El método es una calibración por mínimos y máximos. Al iniciarla desde la interfaz, durante 20 segundos se gira el guante lentamente en todas las direcciones y se registran el mínimo y el máximo de cada eje. La calibración no bloquea el programa, los extremos se acumulan en `magLeer()` con cada paquete, y al cumplirse el tiempo se calcula el resultado (Código 16).

```
1 static void magActualizarCalibracion(float x, float y, float z) {
2   float v[3] = {x, y, z};
3   for (int i = 0; i < 3; i++) {
4     if (v[i] < magMin[i]) magMin[i] = v[i];
5     if (v[i] > magMax[i]) magMax[i] = v[i];
6   }
7   if (millis() - magInicioCal >= MAG_TIEMPO_CAL) magFinalizarCalibracion();
8 }
```

Código 16: Acumulación de extremos durante la calibración.

Al finalizar, el offset de cada eje es el punto medio entre su mínimo y su máximo, lo que corrige el efecto *hard-iron* al centrar la esfera de mediciones en el origen.

La escala se obtiene igualando el radio de cada eje con el radio promedio, una corrección simplificada del efecto *soft-iron* que convierte el elipsoide medido en una esfera. Si algún radio resulta menor a 100 cuentas, se descarta la calibración y se avisa por el puerto serie que no se giró lo suficiente (Código 17).

```
1 for (int i = 0; i < 3; i++) {
2   magOffset[i] = (magMax[i] + magMin[i]) / 2.0f;   // hard-iron
3   radio[i]     = (magMax[i] - magMin[i]) / 2.0f;
4 }
5 if (radio[0] < 100 || radio[1] < 100 || radio[2] < 100) return;  // giro insuficiente
6
7 float radioProm = (radio[0] + radio[1] + radio[2]) / 3.0f;
8 for (int i = 0; i < 3; i++) magScale[i] = radioProm / radio[i];  // soft-iron
```

Código 17: Cálculo de offsets y escalas.

Cada lectura posterior se corrige restando el offset y multiplicando por la escala de su eje, antes de calcular el rumbo.

Mientras dura la calibración el magnetómetro no actualiza sus valores, por lo que el yaw no es confiable durante esos 20 segundos. Además, la calibración debe repetirse en cada vez que se inicia el sistema y el resultado depende de que el guante se gire lejos de objetos metálicos o imanes.

4.8. Estado general

En la Tabla 3 se muestra el resumen de los avances de esta etapa.

Tabla 3: Síntesis del avance experimental.

Actividad	**Estado**
Montaje sobre el guante	Pendiente
Transmisión Wi-Fi	Completado
Calibración de offsets del DMP	Completado, pendiente incorporarlo a la interfaz
Implementación y ajuste del filtro Kalman	En proceso
Optimización del filtro complementario	Completado
Visualización 3D	Completado
Panel selector	Completado
Gráfico de error	Completado
Calibración del magnetómetro	Completado

5. Documentación relacionada

El código fuente está disponible en el repositorio del proyecto, la bitácora de trabajo en la wiki del repositorio y el video de demostración en Google Drive.